

Data Structure (week 5)

講師：陳俊豪

Array 統整

	Static Array	Dynamic Array
對應 Python 結構	Numpy	List
優點	固定記憶體 $O(1)$	彈性記憶體 $O(1)$
缺點	固定記憶體 + $O(N)$ 搬家問題	$O(N)$ 搬家問題

- $O(N)$ 這種搬家問題, 都是牽扯 **insert** 與 **remove** 為主, 而非 **append** 與 **pop**。(p.s. append 雖然在 dynamic array 中會發生 $O(N)$ 但因為久久發生一次, 平攤後仍為 $O(1)$)
- **Worst Case** 才是我們關心的 !

Linked List

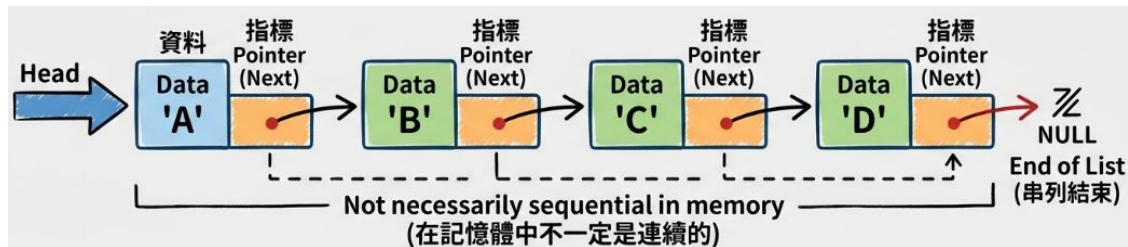
列表 > 串列

Recall : Array 的缺點

- 當資料量 N 很大時 (ex. 100 萬筆), 因為 Array 實體記憶體必須**連續**, 進行 insert 等指令時, 會導致後面的 100 萬筆資料必須全部往後退一步, 產生了 $O(N)$ 的「平移地獄」(搬家)
- 解決方法 :
 - 記憶體真的需要連續嗎, 如果你知道總是記憶體下一個位置在哪呢? (想像牽手)
 - 在 1 號牽 2 號, 2 號牽 3 號, 依此類推下。如果我今天要放入一筆資料在 1 號和 2 號中間, 會發生什麼事? 想想看還會有 $O(N)$ 平移地獄問題嗎?

Linked List (鏈結串列, 鏈表)

- 記憶體不排隊了：
 - 跟作業系統要記憶體時, 不再要求「連續」的大空間, 而是哪裡有空位就塞哪裡。
 - 為了不讓資料走失, 每一筆資料都要額外負責一件事:「記住下一個人的地址」。



- Linked List
 - Node 架構: Data + Next Address (下一個記憶體位置, Pointer)
 - **優點**: 解決了 $O(N)$ 平移地獄, 變回 $O(1)$
 - 但是沒那麼簡單, 下一頁詳述

Linked List :

- Insert 的動作其實都包含 2 步 : Search + Insert

資料結構	Search	Insert	總結
Array	$O(1)$	$O(N)$	找得快, 但搬家慢
Linked List	$O(N)$	$O(1)$	找得慢, 但不用搬家

- 那 Linked List 就只是個 trade-off 關係嗎 ? 合起來都是 $O(N)$
 -
 -

Linked List :

- Insert 的動作其實都包含 2 步 : Search + Insert

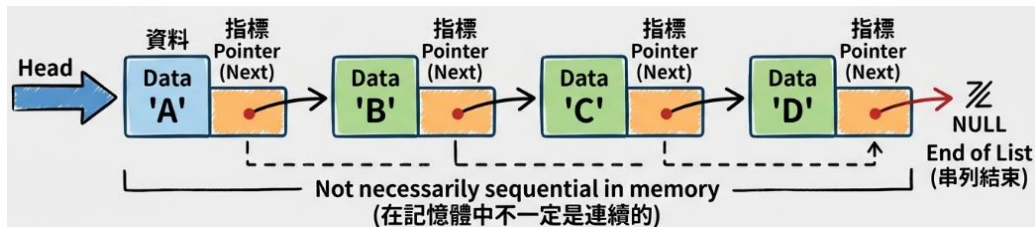
資料結構	Search	Insert	總結
Array	$O(1)$	$O(N)$	找得快, 但搬家慢
Linked List	$O(N)$	$O(1)$	找得慢, 但不用搬家

- 那 Linked List 就只是個 trade-off 關係嗎 ? 合起來都是 $O(N)$
 - Head & Tail 為 $O(1) + O(1) = O(1)$
 - 連續操作 insert : $O(1) + O(N) * n$ v.s. $O(N) + O(1) * n$

Linked List (鏈結串列, 鏈表)

- 記憶體不排隊了：

- 跟作業系統要記憶體時, 不再要求「連續」的大空間, 而是哪裡有空位就塞哪裡。
- 為了不讓資料走失, 每一筆資料都要額外負責一件事：「記住下一個人的地址」。

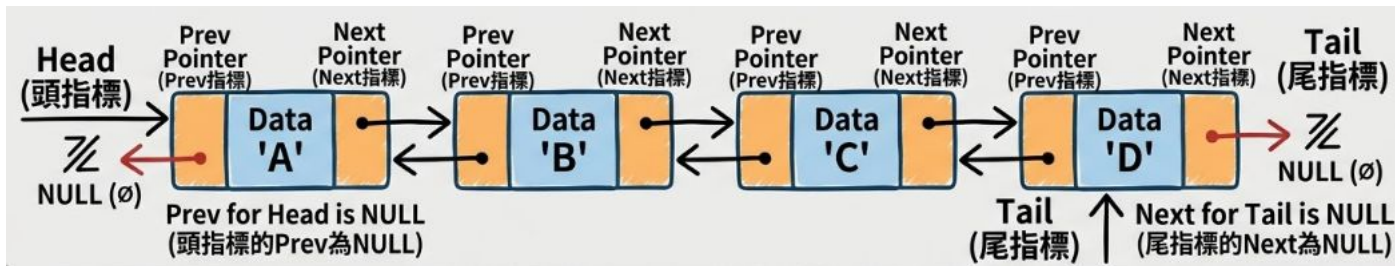


- Linked List

- Node 架構：Data + Next Address (下一個記憶體位置, Pointer)
- **優點**：解決了 $O(N)$ 平移地獄, 變回 $O(1)$ 。不用搬家
- **缺點**：
 - Search 階段變成 $O(N)$
 - 空間上的額外浪費, 每個資料點都要多記「Next Address」。同時失去 CPU Cache 優勢
 - 人生沒有回頭路... 永遠只知道下一個是誰。為了找上一個居然變成 $O(N)$

Doubly Linked List (雙向鏈結串列)

- 記憶體不排隊了：
 - 每個人不但記住「下一個人」是誰，還要記住「上一個人」是誰 (Prev Pointer)
 - 成本增加 (要記住的 address 變成 2 個)



- Doubly Linked List :
 - 優點：
 - 找「上一筆」資料瞬間變成 $O(1)$ ，對應很多 "真實使用情境" ex. 上一頁、上一首、還原
 - 刪除特定資料更方便。在單向串列中，要讓刪除 B，必須讓 A 放開 B 改牽 C。但如果現在就站在 B 身上，根本不知道 A 在哪裡！而雙向串列中，B 左牽 A，右牽 C，B 只要直接叫 A 和 C 互相牽就結束。

In-class exercises 1

- 1. 如果你今天要設計一個「音樂播放清單」(常常要切換下一首, 也偶爾會插播), 請問你怎麼選擇資料結構 (Static Array / Dynamic Array / Linked List) ?
 - **Doubly Linked List**
- 2. 如果你今天要設計一個「全班成績單」(常常要看第 15 名、第 30 名 是誰), 請問怎麼選擇資料結構 ?
 - **Static Array**
 - **Ans = $O(N)$**
 - 理由 :
 - 為了騰出最前面的空間, 原本 100 萬筆資料的地址全部都必須往後 shift 一格。且資料越大花的時間就越多, 因此為 $O(N)$

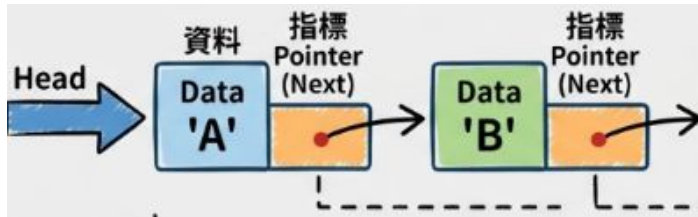
In-class exercises 2

- 3. 你正在帶領團隊開發一款全新的網頁瀏覽器 (ex. Chrome)。現代使用者極度沒有耐心，他們在瀏覽網頁時，常常會瘋狂點擊「上一頁」和「下一頁」，並且要求瞬間切換，絕對不能卡頓。假設現在使用者的電腦記憶體 (RAM) 通常都很充足，各別分析 Singly Linked List 和 Doubly Linked List ?
 - Singly Linked List
 - 「下一頁」沒問題，但「上一頁」會卡死。
 - 但是有省 RAM 優勢！可能任何低階手機都很流暢。
 - Doubly Linked List
 - 「上一頁」和「下一頁」都沒問題。
 - 但是費 RAM 可能快 2 倍，低階手機不適用。

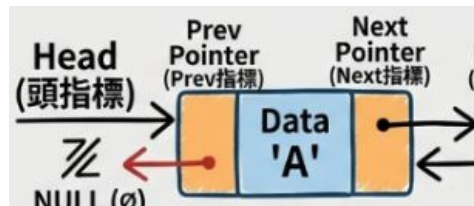
In-class exercises 3

- 4. **低成本物聯網 (IoT) 智慧溫度感測器情境**。因為物聯網裝置一般都很小 (硬體受限, RAM 很小), 目前有一個測量裝置運作邏輯為 :「每分鐘記錄一次溫度, 累積一天後, 半夜 2 點固定把資料上傳到雲端主機, 然後清空」?
 - Singly Linked List

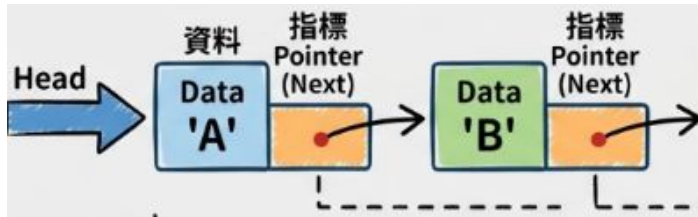
真實世界案例 1：區塊鏈 (Blockchain)



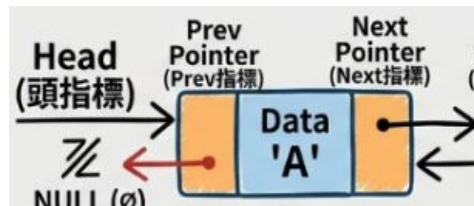
- Blockchain 本身就是一個大型 Singly Linked List (單向)
- Blockchain 保存的是交易資料，想像成任兩人有交易就牽起手，只是原本保存「下一個地址」的 Pointer 改成保存「上一筆交易的數位簽名」(Hash)！
 - Data (交易紀錄) + Pointer (上一筆交易的數位簽名)
 - 「不可竄改」特性：只要檢查「前一筆的 Pointer」與「當前這筆的 Data 內資訊」，就能確定 Data 內的資訊有沒有被竄改，若不一樣就知道這筆資料是假的。
 - 這就是區塊鏈「Trust」的底層來源！
- 為什麼不是 Doubly Linked List？



真實世界案例 1：區塊鏈 (Blockchain)



- Blockchain 本身就是一個大型 Singly Linked List (單向)
- Blockchain 保存的是交易資料，想像成任兩人有交易就牽起手，只是原本保存「下一個地址」的 Pointer 改成保存「上一筆交易的數位簽名」(Hash)！
 - Data (交易紀錄) + Pointer (上一筆交易的數位簽名)
 - 「不可竄改」特性：只要檢查「前一筆的 Pointer」與「當前這筆的 Data 內資訊」，就能確定 Data 內的資訊有沒有被竄改，若不一樣就知道這筆資料是假的。
 - 這就是區塊鏈「Trust」的底層來源！
- 為什麼不是 Doubly Linked List？
 - 交易當下還不知道 Next Pointer 要填什麼 (尚未發生)
 - 分散式系統下單向效能更高，管理方便
 - 密碼學悖論 (補充)



真實世界案例 2：金融指標計算 ex.移動平均線 (MA)

- 金融交易上常看到 MA 等技術指標，運作邏輯為：
 - 假設正在計算「日資料的 5 天 MA」，現在時間過了一天，系統需要把新資料 append 到開後 (最新)，並且把最後一天 remove 掉 (最舊資料)。
 - Array 會產生的問題：
 - 每天都要搬一次家 $O(N)$
 - Linked List 能怎麼解決：
 - 剛剛說到操作「頭尾」是 Linked List 優勢 $O(1)$
 - 因此很多指標在更新時，都只需要改頭尾資料：
 - 最舊 > [Day 1] > [Day 2] > [Day 3] > [Day 4] > [Day 5] > 最新
 - 刪掉 Day 1 : $O(1)$
 - 加入 Day 6 : $O(1)$

Linked List in Python

- Python 內建延伸模組 collections 內的 deque 可以實現 Doubly Linked List
 - deque 是 C-based module (發音念做 “deck” ! 因為就像 a deck of cards)
 - 未來教 Queue、Stack 也會用 deque, 他們是共用的結構 (不用記那麼多也好 !)
- 實際操作
 - `from collections import deque`
 - **# 宣告 Linked List**
 - `linked_ls = deque(["訂單1", "訂單2", "訂單3"])`
 - **# 存取資料 : $O(N)$** (語法和 Numpy 相同, 但不是 $O(1)$ 喔 !)
 - `linked_ls[2]` # "訂單3"
 - `linked_ls[0]` # "訂單1"

Linked List in Python

- 實際操作

- **# Insert : Search $O(N)$ + Insert $O(1)$ = $O(N)$, 優勢在多次操作**
- `linked_ls.insert(1, "訂單VIP")`
- **# 頭尾操作 : 頭部 insert + remove $O(1)$**
- `linked_ls.appendleft("訂單0")`
- `linked_ls.popleft()`
- **# 頭尾操作 : 尾部 insert + remove $O(1)$**
- `linked_ls.append("訂單4")`
- `linked_ls.pop()`
- **# 往回找資料 : Python 沒有**
- 在 C/C++ 中可以把 Pointer 變成變數, 去找 prev 和 next 位置

Array + Linked List 統整表

- 沒有完美的資料結構，只有時間與空間的 trade-off：

	Static Array	Dynamic Array	Singly Linked List	Doubly Linked List
記憶體配置	連續	連續 (會搬家)	散落各處 (會牽手)	散落各處 (多長一隻手)
空間	固定	彈性 (會搬家)	彈性	彈性 (增加成本)
存取資料	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Insert 問題	$O(N)$	$O(N)$	$O(1)$	$O(1)$
往回找資料	$O(1)$	$O(1)$	$O(N)$	$O(1)$



HW2: Static Array (Numpy) + Linked List (Deque)

- 這份程式碼將會模擬大資料客戶的運算。請完成所有 TODO 區塊 (1-1、1-2、2-1、2-2), 並觀察終端機印出的執行時間, 最後用註解回答 3-1、3-2、3-3。
- 這份程式碼模擬了 30 萬人的演唱會排隊系統。我們將測試「VIP 頭部插隊」與「隨機抽查」兩種情境。請完成 TODO 區塊 (4-1、4-2、5-1、5-2), 並觀察 deque (Doubly Linked List) 在不同情況的表現, 最後用註解回答 6-1、6-2。

補充資訊

MLP

